

التوثيق الشامل لمتطلبات مشروع SmartHotel

(A to Z)

هذا الملف مخصص كمرجع تفصيلي للمناقشة الجامعية. يشرح هذا الملف كيف تم تنفيذ كل متطلب من المتطلبات الرسمية تقنياً داخل المشروع من البداية (Frontend) إلى النهاية (Database).

فهرس المتطلبات

1. إدارة المستخدمين والمصادقة (User Management)
2. استعادة كلمة المرور (Password Recovery)
3. نظام الإدارة (Admin) وإدارة الفنادق والغرف (Manager)
4. البحث الذكي ونظام التوصية (Search & Recommendation)
5. مقارنة الفنادق (Hotel Comparison)
6. الخرائط والأماكن السياحية والخدمات القريبة (Maps & Nearby Services)
7. حاسبة تكلفة الرحلة (Trip Cost Calculator)
8. قائمة المفضلة (Favorites)
9. نظام التقييمات (Reviews)
10. نظام الحجوزات (Booking)
11. الأسعار الديناميكية والعروض الفورية (Dynamic Pricing & Flash Deals)
12. مقارنة المنافسين (Competitor Benchmarking)
13. برنامج الولاء والمكافآت (Loyalty Program)
14. دعم العملات والضرائب (Currency & Taxes)
15. معلومات المدينة (City Information) - غير منفذ
16. البنية التحتية، الأمان، و Docker

REQ-01 — إدارة المستخدمين والمصادقة (User)

(Management

1. ما هو المطلوب؟

إنشاء حساب جديد، تسجيل الدخول، تسجيل الخروج، تعديل الملف الشخصي، تغيير كلمة المرور، وحذف الحساب.

2. فكرة عمل الميزة

نظام مصادقة متكامل يسمح للمستخدم بإنشاء حساب باستخدام بريده الإلكتروني وكلمة مرور، ويستخدم JSON (JWT (Web Token لإدارة الجلسات بشكل آمن بدون الحاجة لتخزين الجلسة على الخادم (Stateless).

3. نقطة البداية — Frontend

الملف: `frontend/src/pages/Register.jsx` و `frontend/src/pages/Login.jsx` و `frontend/src/pages/Profile.jsx` ال `State`: `formData` (تحتوي على `email`, `password`, `full_name`, `role`). ال `Hook/Context`: `AuthContext.jsx` يوفر الدالة `login(token, user)` و `logout()`.

4. ماذا يحدث عند تفاعل المستخدم؟ (تسجيل الدخول كمثال)

المستخدم يضغط "Login" ↓ دالة `handleSubmit()` تمنع التحديث الافتراضي (`e.preventDefault()`). ↓ استدعاء `api.post('auth/login', formData)`. ↓ يستقبل ال Backend الطلب ويتحقق من البيانات. ↓ إذا نجح، يعيد ال Backend `token` و `user`. ↓ تقوم `AuthContext` بحفظ ال `token` في `localStorage` وتحديث ال `State`. ↓ يتم توجيه المستخدم (Navigate) إلى الصفحة الرئيسية أو لوحة التحكم حسب دوره (Role).

5. شرح API Request

- **Endpoint**: `POST /api/auth/login`
- **Body**: `{ email, password }`

6. شرح Backend بالتفصيل

المسار: `backend/src/routes/authRoutes.js` يتم توجيه الطلب إلى `login` controller.

ال **Controller**: `backend/src/controllers/authController.js` (دالة `login`)

1. يقرأ `email` و `password` من `req.body`.
2. يبحث عن المستخدم باستخدام `User.findOne({ where: { email } })`.
3. يتحقق من كلمة المرور باستخدام `bcrypt.compare(password, user.password_hash)`.
4. إذا تطابقت، يقوم بإنشاء JWT Token باستخدام `jwt.sign`.
5. يعيد ال Token وبيانات المستخدم في ال `Response`.

7. شرح قاعدة البيانات

- الجدول: `users`
- ال **Model**: `backend/src/models/User.js`
- أهم الحقول: `id`, `full_name`, `email`, `password_hash`, `role`, `security_question_1`, `security_answer_1`

Data Flow (رحلة البيانات)

User Input → `Login.jsx` State → `api.post` → Express Route `/login` → `authController.js` → `Sequelize User.findOne()` → SQL Server → `bcrypt check` → `Generate JWT` → `Response` → `AuthContext` → `localStorage` → UI Update

Example

مستخدم يدخل `user@example.com` و `123456`. ال Backend يقرأ ال hash من قاعدة البيانات (`...$2b$10$`), يقارنه بـ `123456`. إذا تطابق، يصدر Token وينجح الدخول.

كيف أشرح هذا المتطلب للدكتورة؟

"نظام المصادقة يعتمد على JWT. عندما يقوم المستخدم بتسجيل الدخول، يقوم ال Backend بمقارنة كلمة المرور المشفرة باستخدام مكتبة bcrypt. إذا كانت صحيحة، يتم توليد Token وإرساله لل Frontend ليتم حفظه في localStorage واستخدامه في الطلبات اللاحقة عبر ال Headers."

الملفات التي أفتحها أثناء المناقشة

1. frontend/src/context/AuthContext.jsx (لإظهار كيف نحفظ ال Token).

2. backend/src/controllers/authController.js (دالة login لإظهار استخدام bcrypt و jwt).

أسئلة محتملة

السؤال: كيف تحمي كلمات المرور في قاعدة البيانات؟ **الجواب:** لا نقوم بتخزين كلمات المرور كنص صريح (Plain Text). نستخدم مكتبة bcrypt لعمل Hashing لكلمة المرور قبل حفظها (في دالة register). وعند تسجيل الدخول نستخدم bcrypt.compare للتحقق.

REQ-02 — استعادة كلمة المرور (Password Recovery)

1. ما هو المطلوب؟

استعادة كلمة المرور في حال نسيانها، دون استخدام الإيميل أو الرسائل النصية.

2. فكرة عمل الميزة

نظام يعتمد على "الأسئلة الأمنية". عند التسجيل، يختار المستخدم سؤالين من أصل 4 ويجب عليهما. عند استعادة كلمة المرور، يطلب منه النظام الإجابة على نفس السؤالين اللذين اختارهما.

3. نقطة البداية — Frontend

الملف: `frontend/src/pages/Login.jsx` (وضع Forgot Password). العملية: يدخل المستخدم بريده الإلكتروني → النظام يجلب أسئلته الأمنية → يجيب عليها → يكتب كلمة المرور الجديدة.

4. شرح Backend بالتفصيل (Controller)

الملف: `backend/src/controllers/authController.js` هناك 3 دوال أساسية:

1. `forgotPassword`: تستقبل الـ `email`، تبحث عن المستخدم، وتعيد `security_question_1` و `security_question_2` (بدون الإجابات).
2. `verifySecurityAnswers`: تستقبل الـ `email` والإجابات. تقارن الإجابات مع `security_answer_1` و `security_answer_2` (باستخدام تحويل النصوص لـ `Lowercase` لتجنب أخطاء حالة الأحرف). إذا نجحت، تعيد `resetToken` مؤقت.
3. `resetPassword`: تستقبل الـ `resetToken` و `newPassword`. تقوم بتشفير كلمة المرور الجديدة وتحديث قاعدة البيانات.

كيف أشرح هذا المتطلب للدكتورة؟

"بدلاً من إرسال إيميل، قمت ببرمجة نظام أسئلة أمنية. المستخدم يختار سؤالين عند التسجيل. عند النسيان، يسترجع الـ Backend أسئلته المحددة من قاعدة البيانات ويعرضها له. إذا أجاب بشكل صحيح، يحصل على Token مؤقت يسمح له بتغيير كلمة المرور."

الملفات التي أفتحها أثناء المناقشة

1. `backend/src/controllers/authController.js` (دوال `forgotPassword` و `verifySecurityAnswers`).

REQ-03 — نظام الإدارة (Admin) وإدارة الفنادق والغرف (Manager)

1. ما هو المطلوب؟

مدير النظام (Admin) ينشئ مدراء الفنادق. مدير الفندق (Hotel Manager) يدير فندقه، يضيف الغرف، يحدد السعر والحالة. ملاحظة: "إدارة صور الفندق والغرف" و "تغيير حالة الغرفة يدوياً" هي ميزات منقوصة جزئياً في الـ UI ولكنها مدعومة في قاعدة البيانات.

2. فكرة عمل الميزة

اعتمدنا معمارية صارمة: كل فندق يملكه مدير واحد فقط، وكل مدير يملك فندقاً واحداً فقط. بدل أن يقوم الـ Admin بإنشاء الفندق ثم إنشاء المدير ثم ربطهما، قمنا بدمج العملية في **Transaction** واحد. عندما ينشئ الـ Admin حساب مدير، يقوم النظام آلياً بإنشاء الفندق وربطه به.

3. شرح Backend بالتفصيل (Atomic Creation)

الملف: `backend/src/controllers/adminController.js` (دالة `createUser`)

1. يتم إنشاء `sequelize.transaction()`.
2. يتم إنشاء حساب المدير (User) في الـ `Transaction`.
3. إذا كان دوره `hotel_manager`، يتم فوراً إنشاء سجل في جدول `Hotels` باسم مؤقت (مثلاً "Hotel for X Manager").
4. يتم تحديث `hotel_id` الخاص بالمدير ليرتبط بهذا الفندق.
5. يتم عمل `transaction.commit()`. إذا فشلت أي خطوة، يتم عمل `rollback` ولن يتم إنشاء أي شيء (لا مدير يتيم ولا فندق يتيم).

4. إدارة الغرف (Room Management)

الملف: frontend/src/pages/ManagerPortal.jsx ال Controller:

backend/src/controllers/managerController.js (createRoom , updateRoom ,

deleteRoom). عندما يضيف المدير غرفة، يرسل (نوع الغرفة، السعة، السعر، عدد الغرف المتاحة). يأخذ ال Backend

req.user.hotel_id (لضمان أن المدير يضيف الغرفة لفندقه فقط) ويقوم بـ Room.create().

5. الميزات غير المكتملة (Not Implemented / Partially Implemented) / الميزات المكتملة (Implemented)

- إدارة صور الغرف والفنادق: غير منفذة في واجهة المستخدم (UI)، لا يوجد زر لرفع الصور، لكن قاعدة البيانات تحتوي على حقول primary_image_url .
- تغيير حالة الغرفة: ال Model يحتوي على حقل status ولكن واجهة المدير لا تحتوي على زر لتغيير حالة الغرفة يدوياً.

كيف أشرح هذا المتطلب للدكتورة؟

"لضمان سلامة البيانات (Data Integrity)، جعلت عملية إنشاء الفندق والمدير عملية واحدة (Atomic Transaction). عندما ينشئ الأدمن مدير فندق، يتم إنشاء الفندق برمجياً في نفس اللحظة. واجهة مدير الفندق تتيح له لاحقاً تعديل اسم فندقه وإضافة الغرف وتحديد أسعارها."

الملفات التي أفتحها أثناء المناقشة

1. backend/src/controllers/adminController.js (لإظهار ال sequelize.transaction()).

2. frontend/src/pages/ManagerPortal.jsx (لإظهار كيف يدير المدير فندقه).

REQ-04 — البحث الذكي ونظام التوصية (Smart Search & Recommendation)

1. ما هو المطلوب؟

يتيح النظام للمستخدم البحث عن الفنادق. ثم يقوم بتحليل نتائج البحث باستخدام خوارزمية توصية ذكية تعتمد على تفضيلات المستخدم وتعرض "نسبة توافق" و "سبب التوصية".

2. فكرة عمل الميزة

البحث العادي يصفي الفنادق حسب المدينة والسعر. أما خوارزمية التوصية فتأخذ الفنادق المفترزة وتقوم بحساب Score لكل فندق بناءً على عوامل متعددة (نوع الرحلة، السعر المطلوب، التقييمات، المرافق، العروض)، ثم تحوله إلى نسبة مئوية.

3. شرح البحث (Hotel Search)

الملف: `backend/src/controllers/hotelController.js` (دالة `getHotels`) يستقبل: `city_id`, `min_price`, `max_price`, `star_rating`, `amenities`. يقوم ببناء `whereClause` لـ `Sequelize`. ملاحظة: تصفية الفنادق بناءً على التواريخ (Availability Filter) غير مكتملة في الـ Query الأساسي للبحث، لكن يتم التحقق من التوفر الفعلي عند الحجز.

4. شرح خوارزمية التوصية (Recommendation Algorithm) بالتفصيل

الملف: `backend/src/controllers/recommendationController.js`

المدخلات:

- `trip_type` (business, family, couple)
- `target_price` (الميزانية المطلوبة)
- `amenities` (الخدمات المطلوبة)
- `allHotels` (الفنادق الموجودة في المدينة المحددة - لا نعطي وزن للمدينة لأنها فلترة إجباري).

الخطوات والمعادلات:

يتم حساب `score` يبدأ من صفر لكل فندق:

1. التقييم بالنجوم (**Star Rating**): الوزن 1.5 `score += star_rating * 15` (مثال: 4 نجوم = 60 نقطة).

2. نوع الرحلة (**Trip Type**):

○ إذا كان `business` والفندق يملك (WiFi, Meeting Room) `+25` نقطة.

○ إذا كان `family` والفندق يملك (Pool, Kids Club) `+25` نقطة.

3. تقارب السعر (**Price Proximity**): الوزن 2.0 المعادلة: `priceScore = Math.max(0, 50 -`

`(priceDiff / target_price) * 30)` (كلما اقترب سعر الفندق من ميزانية المستخدم، زاد ال Score بحد أقصى 50).

4. المرافق (**Amenities**): تطابق كل مرفق مطلوب مع مرافق الفندق يضيف 12 نقطة.

5. متوسط التقييمات (**Review Rating**): `score += avgReviewScore * 8`

6. العروض الفورية (**Flash Deals**): إذا كان هناك عرض نشط `+25` نقطة.

الناتج (Output):

- نسبة التوافق: يتم قص ال Score ليكون بحد أقصى 99% (`Math.min(99, score)`).
- سبب التوصية (**Match Reason**): أثناء الحساب، إذا حقق الفندق شرطاً عالياً (مثلاً السعر قريب جداً)، نضيف للنص: `"Great match for your target budget"`.
- يتم ترتيب المصفوفة تنازلياً حسب ال Score.

كيف أشرح هذا المتطلب للدكتورة؟

"بدل أن أعتد على ترتيب عشوائي، برمجت Recommendation Engine في ال Backend. الخوارزمية تحسب سكور لكل فندق. لم أضع المدينة كعامل وزن لأن المستخدم اختارها مسبقاً. الخوارزمية تقارن ميزانية المستخدم بسعر الفندق بمعادلة رياضية، وتفحص نوع الرحلة (مثلاً رحلة عائلية تبحث تلقائياً عن وجود مسبح). النتيجة النهائية تظهر للمستخدم كنسبة تطابق (Match Percentage) مع توضيح سبب التوصية."

الملفات التي أفتحها أثناء المناقشة

1. `backend/src/controllers/recommendationController.js` (شرح ال Math والحسابات).

2. `frontend/src/pages/Hotels.jsx` (لإظهار كيف نعرض ال Match %).

REQ-05 — مقارنة الفنادق (Hotel Comparison)

1. ما هو المطلوب؟

اختيار فنادق للمقارنة، عرض جدول للمقارنة (السعر، التقييم، الخدمات)، تصدير PDF، حفظ المقارنات السابقة، ومعالجة الحقول الفارغة (غير متوفر).

2. فكرة عمل الميزة

يستخدم النظام `ComparisonContext` في الـ Frontend لحفظ الـ IDs الخاصة بالفنادق المختارة (بحد أقصى 4). يتم إرسال الـ IDs للـ Backend الذي يجمع بيانات الفنادق ويبنى "مصفوفة مرافق" (Amenity Matrix) لتوحيد الجدول.

3. شرح Backend بالتفصيل (Comparison Matrix)

الملف: `backend/src/controllers/comparisonController.js` (دالة)

(`getSideBySideComparison`)

- يستقبل مصفوفة `hotel_ids`.
- يجلب الفنادق عبر Sequelize مع جميع العلاقات (Rooms, Amenities, Reviews, NearbyServices).
- معالجة الحقول الفارغة: يقوم الـ Backend بإنشاء `amenityMap` يجمع كل المرافق الموجودة في كل الفنادق المقارنة. ثم يمر على كل فندق؛ إذا كان الفندق لا يملك المرفق، يضع له `{ available: false }` لكي يظهر في الـ Frontend كعلامة "X" أو "غير متوفر".

4. تصدير PDF وحفظ المقارنة

- **PDF Export:** تم تنفيذه في `frontend/src/pages/Compare.jsx` باستخدام مكتبة `html2pdf.js`. تقوم المكتبة بأخذ عنصر الـ HTML (الجدول) وتحويله إلى Canvas ثم إلى PDF.
- **Save Comparison:** يتم إرسال الطلب إلى `POST /api/comparison/saved` لحفظ مصفوفة الـ IDs في جدول `saved_comparisons` الخاص بالمستخدم للرجوع إليها لاحقاً.

كيف أشرح هذا المتطلب للدكتورة؟

"في المقارنة، التحدي الأكبر كان معالجة المرافق غير المتوفرة. برمجت في ال Backend منطقاً يبنّي (Matrix). إذا كان فندق (أ) يملك مسبح وفندق (ب) لا يملك، ال Backend يضيف المسبح لفندق (ب) ولكن بحالة (available: false) لتظهر واجهة المستخدم إشارة خطأ حمراء بوضوح. كما وفرت زر تصدير يعتمد على مكتبة html2pdf لالتقاط الجدول وتنزيله."

الملفات التي أفتحتها أثناء المناقشة

1. backend/src/controllers/comparisonController.js (سطر amenityMap).

2. frontend/src/pages/Compare.jsx (دالة handleExportPDF).

REQ-06 — الخرائط والأماكن السياحية والخدمات القريبة

1. ما هو المطلوب؟

عرض الفندق على الخريطة، وعرض الخدمات القريبة (مطاعم، صيدليات، الخ) والأماكن السياحية، وحساب المسافة بينها وبين الفندق.

2. فكرة عمل الميزة

بدلاً من إدخال الخدمات يدوياً في قاعدة البيانات، قمت بدمج واجهة **OpenStreetMap (Overpass API)** لجلب الخدمات الحقيقية القريبة من إحداثيات الفندق (Latitude/Longitude) بشكل مباشر.

3. شرح ال Frontend والتكامل مع API

الملف: frontend/src/pages/HotelDetail.jsx ال Component يعتمد على react-leaflet لعرض الخريطة.

جلب البيانات (Overpass API):

```
const query = `[out:json][timeout:25];
(
  node["tourism"~"attraction|museum|viewpoint"](around:2500,${lat},${lon});
  node["amenity"~"restaurant|cafe|hospital|pharmacy|atm"](around:2500,${lat},${lon});
);
out body;`;
```

يتم إرسال الإحداثيات، ويعيد الـ API الأماكن ضمن قطر 2.5 كيلومتر.

4. حساب المسافة (Haversine Formula)

لحساب المسافة الدقيقة بين الفندق والمكان القريب، استخدمت خوارزمية **Haversine** المكتوبة يدوياً في الملف:

```
function getDistanceFromLatLonInKm(lat1, lon1, lat2, lon2) {
  const R = 6371; // Radius of the earth in km
  // ... Math.sin, Math.cos ...
  return R * c; // Distance in km
}
```

كيف أشرح هذا المتطلب للدكتورة؟

"لعرض الخدمات القريبة، استخدمت خريطة Leaflet وربطتها بـ OpenStreetMap API. الخريطة تأخذ إحداثيات الفندق وتجلب المطاعم والصيدليات الحقيقية في دائرة قطرها 2.5 كم. ثم استخدمت معادلة Haversine الرياضية في الكود لحساب المسافة الدقيقة بالكيلومتر بين الفندق وكل نقطة خدمة وعرضها للمستخدم."

الملفات التي أفتحها أثناء المناقشة

1. `frontend/src/pages/HotelDetail.jsx` (معادلة `getDistanceFromLatLonInKm` واستعلام `Overpass API`).

REQ-07 — حاسبة تكلفة الرحلة (Trip Cost Calculator)

1. ما هو المطلوب؟

حساب تكلفة الفندق، الطعام، المواصلات بناءً على عدد الأيام وعدد الأشخاص (البالغين والأطفال)، وعرض التكلفة الإجمالية.

2. فكرة عمل الميزة

صفحة تفاعلية تتيح للمستخدم تحديد الفندق والغرفة وعدد الأشخاص والتواريخ. النظام يقوم بحساب التكلفة اعتماداً على أسعار الغرفة مضافاً إليها التكلفة المعيشية للمدينة (المخزنة في قاعدة البيانات).

3. شرح الخوارزمية (Calculations)

الملف: `frontend/src/pages/TripCostCalculator.jsx`

- **تكلفة الفندق (Hotel Cost):** $\text{سعر الغرفة في الليلة} \times \text{عدد الغرف} \times \text{عدد الليالي}$
- **أيام الرحلة (Trip Days):** يتم حسابها كـ $\text{عدد الليالي} + 1$ (لأن الطعام يستهلك في يوم المغادرة أيضاً).
- **تكلفة الطعام (Food Cost):** $\text{متوسط تكلفة الطعام في المدينة} \times \text{إجمالي الأشخاص (ببالغين + أطفال)} \times \text{أيام الرحلة}$
- **تكلفة المواصلات (Transport Cost):** $\text{متوسط تكلفة المواصلات في المدينة} \times \text{إجمالي الأشخاص} \times \text{أيام الرحلة}$
- **التكلفة الإجمالية (Total Cost):** مجموع كل ما سبق + أي تكاليف إضافية (نشاطات) يدخلها المستخدم.

كيف أشرح هذا المتطلب للدكتورة؟

"الحاسبة تفاعلية بالكامل بال Frontend. عندما يختار المستخدم فندقاً، يجلب النظام 'متوسط تكلفة الطعام والمواصلات' الخاص بمدينة الفندق من ال Backend. ثم يقوم ال React بحساب التكلفة ديناميكياً كلما غير المستخدم عدد الأطفال أو تواريخ السفر، حيث يضرب تكلفة الشخص في عدد الأيام، ويجمعها مع تكلفة الغرفة ليعطي التكلفة الإجمالية للرحلة."

REQ-08 — قائمة المفضلة (Favorites)

1. ما هو المطلوب؟

إضافة/حذف فندق من المفضلة، عرض القائمة، واستخدامها في التوصيات.

2. فكرة عمل الميزة

في ال Frontend، يوجد زر القلب (Heart Icon). عند الضغط عليه، يتم إرسال طلب إلى `POST /api/favorites/toggle`. يتحقق ال Backend: إذا كان الفندق موجوداً يحذفه، وإذا لم يكن موجوداً يضيفه.

3. الربط مع التوصيات

في `recommendationController.js`، يتم جلب قائمة مفضلة المستخدم. إذا كان الفندق الذي يتم تقييمه يشارك خصائص (مثل Amenities) مع فنادق موجودة في مفضلة المستخدم، يمكن أن يرفع ذلك من دقة التوصية (يتم استخدام ال User Preferences المأخوذة من المفضلة).

REQ-09 — التقييمات (Reviews)

1. ما هو المطلوب؟

إضافة تقييم (نظافة، موقع، خدمة، قيمة)، حساب المتوسط.

2. فكرة عمل الميزة

المستخدم يمكنه ترك تقييم مفصل. ال Database تخزن تقييم كل عنصر من 1 إلى 5. المتوسط يتم حسابه في ال Backend إما برمجياً في ال Controller أو عبر SQL Aggregate Functions.

3. شرح الحساب (Average Rating)

في `hotelController.js`، يتم جلب الفندق مع علاقة الـ `Reviews` الـ Frontend (في `HotelDetail.jsx`) يقوم أحياناً بجمع التقييمات وقسمتها على العدد:

```
const sum = reviews.reduce((acc, r) => acc + Number(r.overall_rating), 0);
const avg = sum / reviews.length;
```

REQ-10 — نظام الحجوزات (Booking)

1. ما هو المطلوب؟

حجز فندق، التحقق من التوفر، حجز غرفتين في عملية واحدة (مدعوم كـ `numRooms`)، تأكيد الحجز، حفظه.

2. فكرة عمل الميزة

أهم خطوة هنا هي ضمان عدم الحجز المتضارب (**Overbooking**) وإدارة الخصومات والضرائب ونقاط الولاء في نفس اللحظة (Transaction).

3. شرح Backend بالتفصيل (Booking Flow)

الملف: `backend/src/controllers/bookingController.js` (دالة `createBooking`)

1. **Transaction & Locking**: يبدأ بـ `sequelize.transaction()` لضمان التزامن.

2. **Availability Check**: يتحقق من الغرفة: `if (room.available_rooms < numRooms) throw .Error`

3. **Price Calculation**: يحسب السعر المبدئي: `basePrice = price_per_night * numRooms * .nights`

4. **Dynamic Pricing** (إن وجد): يبحث عن `DynamicPricingRule` نشطة في نفس التواريخ. إذا وجدها، يضرب السعر بمعامل السعر `multiplier`.

5. **Flash Deals**: يتحقق من العروض. إذا وجد عرضاً نشطاً، يخصم النسبة `promoDiscount`.

6. **Loyalty Vouchers**: إذا أرسل المستخدم `reward_id`، يتحقق الـ Backend أن المستخدم يملك نقاطاً كافية (`UserLoyalty`)، ثم يخصم قيمة المكافأة ويخصم النقاط من رصيده.

7. **Taxes**: يتم إضافة 3% كضريبة: `tax_amount = base_discounted_price * 0.03`.

8. **Finalizing**:

- يخصم عدد الغرف: `room.available_rooms -= numRooms`
- يحفظ الحجز في جدول `Bookings`.
- يضيف نقاط ولاء جديدة للمستخدم بناءً على المبلغ المدفوع.
- ينفذ `transaction.commit()`.

كيف أشرح هذا المتطلب للدكتورة؟

"عملية الحجز معقدة جداً لأنها تحدث داخل Database Transaction واحد. عندما يضغط المستخدم حجز، الـ Backend يقرأ سعر الغرفة، يطبق السعر الديناميكي إذا كان موسماً مزدحماً، يخصم نسبة العروض الفورية، يخصم قيمة قسيمة الولاء إن وجدت، يضيف 3% ضرائب، يتأكد أن الغرفة متاحة وينقص عددها، وأخيراً يمنح المستخدم نقاط ولاء جديدة.. كل هذا في طلب واحد، وإذا فشل أي جزء، يتم التراجع عن كل شيء لضمان عدم حدوث أخطاء مالية."

الملفات التي أفتحها أثناء المناقشة

1. `backend/src/controllers/bookingController.js` (سطور حساب السعر، الخصم، الضريبة، والـ Transaction).

REQ-11 — الأسعار الديناميكية والعروض الفورية (Dynamic & Flash Deals)

1. ما هو المطلوب؟

مدير الفندق يستطيع تعديل الأسعار تلقائياً في مواسم محددة، وإنشاء عروض فورية لتصريف الغرف.

2. فكرة عمل الميزة

مدير الفندق من لوحة التحكم (ManagerPortal) يضيف قاعدة تسعير (Pricing Rule) تحدد تواريخ بداية ونهاية ومعامل (Multiplier). مثلاً 1.5 يعني زيادة 50%. الـ Booking System يتأكد من وجود هذه القواعد أثناء الحجز ليعدل السعر.

3. العروض الفورية (Flash Deals)

نفس المبدأ، يضيف المدير عرضاً بنسبة خصم (مثلاً 20%). يظهر العرض في Frontend باستخدام أيقونات بارزة (Sparkles). عند الحجز، الـ Backend ينقص النسبة من الإجمالي السعر.

REQ-12 — لوحة تحليلات ومقارنة المنافسين (Competitor Benchmarking)

1. ما هو المطلوب؟

تقديم تقارير للمدير تقارن أداء فندقه بمتوسط السوق المحلي في نفس الفئة (بنفس المدينة)، مع ضوابط الخصوصية والأمان.

2. فكرة عمل الميزة

المدير لا يجب أن يرى بيانات الفنادق المنافسة بشكل فردي. لذلك، يقوم الـ Backend بجلب جميع الفنادق في نفس المدينة، ويقوم بجمع الأسعار والتقييمات، ثم يقسمها على العدد (Aggregation) ويعيد المتوسط فقط للمدير.

3. شرح الكود (Aggregation & Privacy)

الملف: `backend/src/controllers/managerController.js` (دالة `getCompetitorBenchmarking`)

1. يجلب فندق المدير ليعرف الـ `city_id`.

2. يجلب المنافسين: `Hotel.findAll({ where: { city_id: myHotel.city_id } })`.

3. يفلتر فندق المدير من القائمة (`otherHotels`).

4. يحسب المتوسطات:

```
marketAvgPrice = otherHotels.reduce((sum, h) => sum + h.base_price_per_night, 0
```

5. يحلل الفرق ويعيد نصيحة للمدير: إذا كان سعر الفندق أقل من السوق بـ 5% يعيد: "أسعارك أقل من السوق، فكر في رفع السعر".

كيف أشرح هذا المتطلب للدكتورة؟

"لتحقيق متطلب مقارنة المنافسين مع الحفاظ على الخصوصية (Privacy)، ال Backend هو من يقوم بكل العمل. يجلب أسعار كل فنادق المدينة من قاعدة البيانات، يدمجها ويحسب المتوسط الحسابي، ويرسل المتوسط فقط للواجهة. بالتالي مستحيل لمدير الفندق أن يعرف سعر فندق منافس بعينه، بل يعرف فقط أين يقع فندقه مقارنة بالسوق."

الملفات التي أفتحها أثناء المناقشة

1. `backend/src/controllers/managerController.js` (دالة `getCompetitorBenchmarking`).

REQ-13 — برنامج الولاء والمكافآت (Loyalty Program)

1. ما هو المطلوب؟

احتساب نقاط الولاء، إضافتها للحساب، استبدالها بمكافآت (قسائم)، استخدامها في الحجز.

2. فكرة عمل الميزة

النقاط في هذا النظام مرتبطة بالفندق المحدد، وليست نقاطاً عامة للمنصة (المدير يتحمل تكلفة الخصم). جدول `user_loyalty` يربط (المستخدم، الفندق، النقاط الحالية). و جدول `loyalty_transactions` يسجل حركات الكسب والصرف.

3. دورة النقاط الكاملة (Data Flow)

كسب النقاط: بعد تأكيد الحجز، يقوم `bookingController` بحساب النقاط (مثال: السعر $\times 10$). ويتم تحديث/إنشاء السجل في `user_loyalty` وإضافة سجل في `loyalty_transactions` بنوع `earned`. إنشاء المكافآت: مدير الفندق يضيف `LoyaltyReward` (مثلاً: 20% خصم مقابل 500 نقطة). استبدال النقاط أثناء الحجز: المستخدم في شاشة الحجز يختار الـ `Reward`. الـ `Frontend` يرسل `reward_id` مع طلب الحجز. في الـ `Backend`:

```
// تأكد من رصيد النقاط
if (userLoyalty.current_points < pendingReward.points_cost) throw Error;

// حساب الخصم
loyaltyDiscount = priceAfterPromo * (percent / 100);

// خصم النقاط
userLoyalty.current_points -= pendingReward.points_cost;
await userLoyalty.save();
```

كيف أشرح هذا المتطلب للدكتورة؟

"برنامج الولاء مصمم ليكون خاصاً بكل فندق. لا أستطيع حجز فندق في دمشق واستخدام نقاط كسبتها في فندق بحلب. يتم تسجيل النقاط كـ `Transactions` (مثل كشف الحساب البنكي). عند الحجز، يستطيع المستخدم تمرير ID القسيمة، ليقوم الخادم بالتحقق من رصيده، خصم النقاط منه، وتطبيق الخصم المالي على الفاتورة النهائية فوراً."

REQ-14 — دعم العملات والضرائب (& Currency Taxes)

1. ما هو المطلوب؟

عرض الأسعار بأكثر من عملة، تحويل العملة، حساب الضرائب والرسوم.

2. فكرة عمل الميزة

يوجد Custom Hook في React اسمه `useCurrency.js` يتفقد العملة المفضلة للمستخدم (EUR أو USD). جميع الأسعار في قاعدة البيانات مخزنة باليورو. ال Hook يضرب السعر بمعدل التحويل (مثال 1.10) قبل عرضه للمستخدم. بالنسبة للضرائب، يتم حسابها وثبيتها في ال Backend (3% Tax) وتُحفظ في جدول `Bookings` (عمود `tax_amount`).

REQ-15 — معلومات المدينة (City Information)

الحالة: غير منفذ / غير مكتمل (UNVERIFIED) السبب: لا يوجد استدعاء ديناميكي لبيانات الطقس (Weather API) في الواجهة الرئيسية. تم ذكر المتطلب في ال PDF ولكن لم يتم بناء الواجهة المخصصة لعرض "أفضل وقت للزيارة والطقس".

البنية التحتية، الأمان، و Docker

1. الأمان (Security)

- **Hashing:** `bcrypt` لتشفير كلمات المرور.
- **Tokens:** `jsonwebtoken` (JWT) للمصادقة وتأمين ال Routes.
- **Authorization:** Middleware `requireRole` يمنع وصول مستخدم عادي لصفحات المدير.
- **Transactions:** Sequelize Transactions لحماية الحجوزات من الانقطاع النصفي.

2. بيئة التشغيل (Docker)

المشروع مجهز للعمل عبر الحاويات (Containers) لضمان بيئة متطابقة:

- **Frontend Container:** مبني باستخدام Nginx لتقديم ملفات React المبنية.
- **Backend Container:** بيئة Node.js.
- **Database Container:** Microsoft SQL Server. يتم تنسيقها عبر `docker-compose.yml`. عند تشغيل `docker compose up`، يتم تفعيل الحاويات وربط الشبكة الداخلية بينها، بحيث يتصل ال Backend بـ `db:1433`.

الخاتمة

تم شرح كافة التفاصيل التقنية اللازمة للمناقشة بناءً على الكود الحقيقي المنفذ في المشروع. يرجى مراجعة الأقسام المحددة وفتح الملفات المذكورة في محرر الأكواد (VS Code) لتوضيح التطبيق العملي أمام لجنة المناقشة.